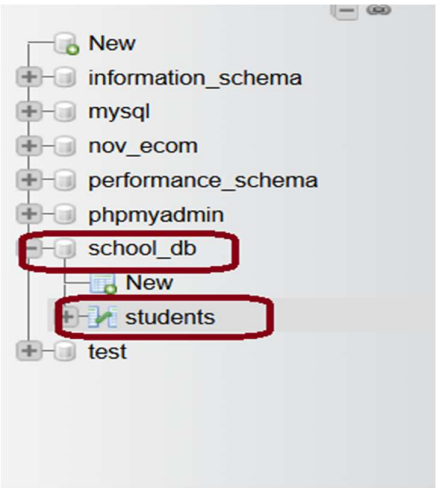
Module 4 – Introduction to DBMS - Practical

1. Introduction to SQL:

Lab 1: Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

Ans:

```
CREATE DATABASE school_db;

USE school_db;

CREATE TABLE students (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(100),
  age INT,
  class VARCHAR(50),
  address VARCHAR(255)
);
```



Lab 2: Insert five records into the students table and retrieve all records using the SELECT statement.

Ans:

Inserting five records:

```
INSERT INTO students (student_id, student_name, age, class, address) VALUES
```

(1, 'Rahul Sharma', 14, '8A', 'Ahmedabad'),

(2, 'Priya Patel', 15, '9B', 'Surat'),

(3, 'Aman Verma', 13, '7C', 'Vadodara'),

(4, 'Sneha Singh', 14, '8B', 'Rajkot'),

(5, 'Karan Mehta', 15, '9A', 'Bhavnagar');

Retrieving all records:

SELECT * FROM students;

	student_id	student_name	age	class	address
Edit Copy Delete	1	Rahul Sharma	14	8A	Ahmedabad
Edit Copy Delete	2	Priya Patel	15	9B	Surat
Edit Copy Delete	3	Aman Verma	13	7C	Vadodara
Edit Copy Delete	4	Sneha Singh	14	8B	Rajkot
Edit Copy Delete	5	Karan Mehta	15	9A	Bhavnagar

Check all *With selected:* Edit Copy Delete Export

2. SQL Syntax:

Lab 1: Write SQL queries to retrieve specific columns (student_name and age) from the students table.

Ans:

SELECT student_name, age FROM students;

	student_name	age
Edit Copy Delete	Rahul Sharma	14
Edit Copy Delete	Priya Patel	15
Edit Copy Delete	Aman Verma	13
Edit Copy Delete	Sneha Singh	14
Edit Copy Delete	Karan Mehta	15

Lab 2: Write SQL queries to retrieve all students whose age is greater than 10.

Ans:

```
SELECT * FROM students WHERE age > 10;
```

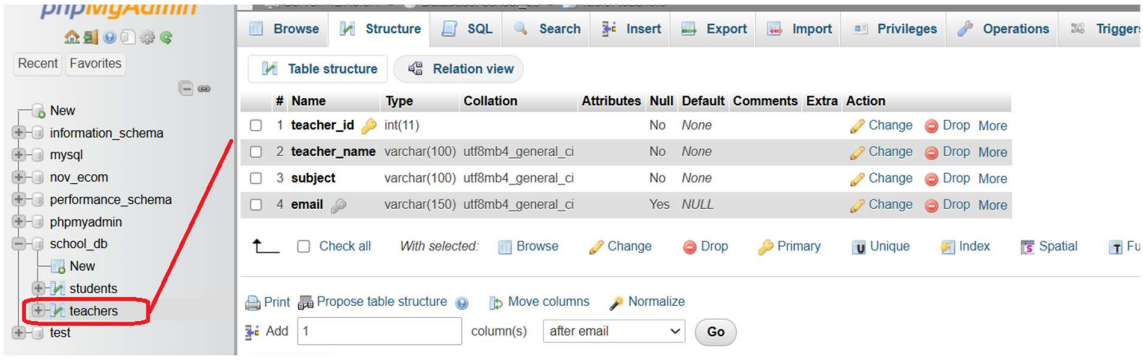
student_id	student_name	age	class	address
1	Rahul Sharma	14	8A	Ahmedabad
2	Priya Patel	15	9B	Surat
3	Aman Verma	13	7C	Vadodara
4	Sneha Singh	14	8B	Rajkot
5	Karan Mehta	15	9A	Bhavnagar

3. SQL Constraints:

Lab 1: Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

Ans:

```
CREATE TABLE teachers (
  teacher_id INT PRIMARY KEY,
  teacher_name VARCHAR(100) NOT NULL,
  subject VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE
);
```



Lab 2: Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

Ans:

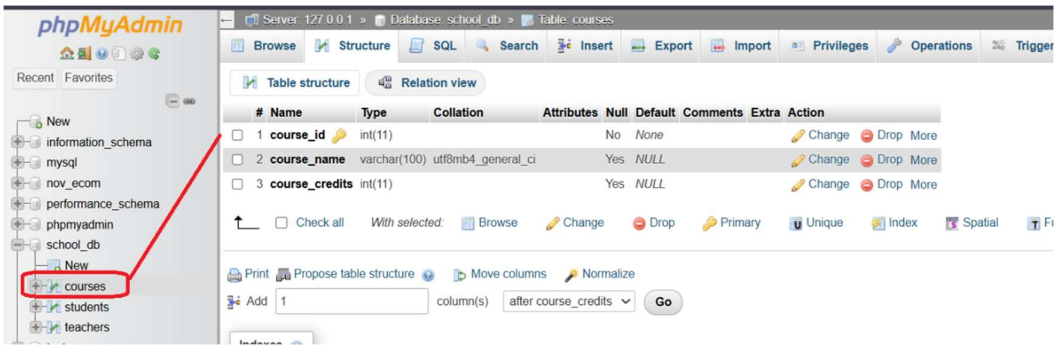
```
ALTER TABLE students ADD teacher_id INT;


ALTER TABLE students
ADD CONSTRAINT fk_teacher
FOREIGN KEY (teacher_id)
REFERENCES teachers(teacher_id);
```

4. Main SQL Commands and Sub-commands (DDL):

Lab 1: Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

Ans:

```
        CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100),
  course_credits INT
);
```



Lab 2: Use the CREATE command to create a database university_db.

Ans:

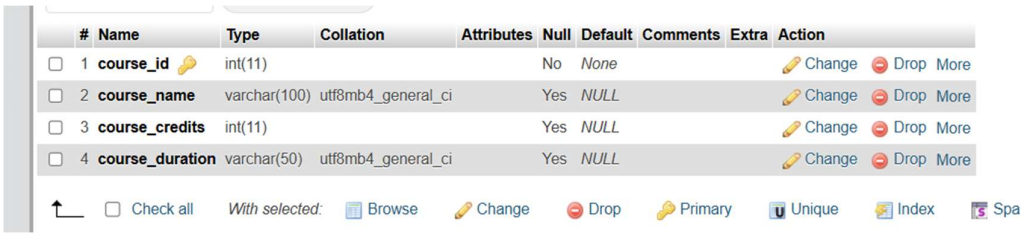
```
CREATE DATABASE university_db;
```

5. ALTER Command:

Lab 1: Modify the courses table by adding a column course_duration using the ALTER command.

Ans:

ALTER TABLE courses ADD course_duration VARCHAR(50);



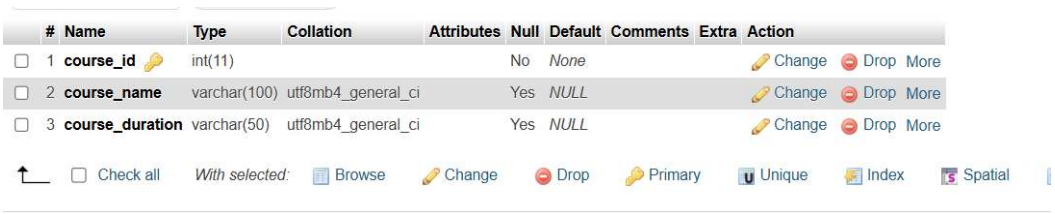
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	course_id	int(11)			No	None			Change Drop More
2	course_name	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	course_credits	int(11)			Yes	NULL			Change Drop More
4	course_duration	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Check all With selected: Browse Change Drop Primary Unique Index Spa

Lab 2: Drop the course_credits column from the courses table.

Ans:

ALTER TABLE courses DROP COLUMN course_credits;



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	course_id	int(11)			No	None			Change Drop More
2	course_name	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	course_duration	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

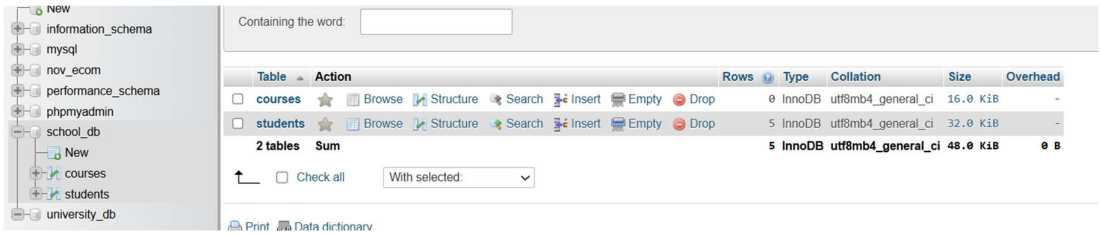
Check all With selected: Browse Change Drop Primary Unique Index Spatial

6. DROP Command:

Lab 1: Drop the teachers table from the school_db database.

Ans:

DROP TABLE teachers;

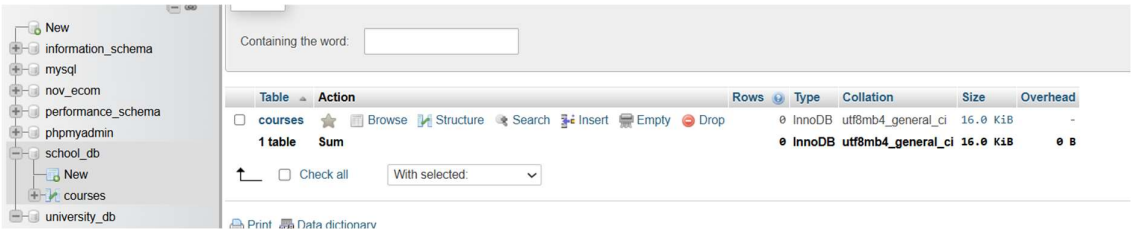


Lab 2: Drop the students table from the school_db database and verify that the table has been removed.

Ans: USE school_db;

DROP TABLE students;

SHOW TABLES;

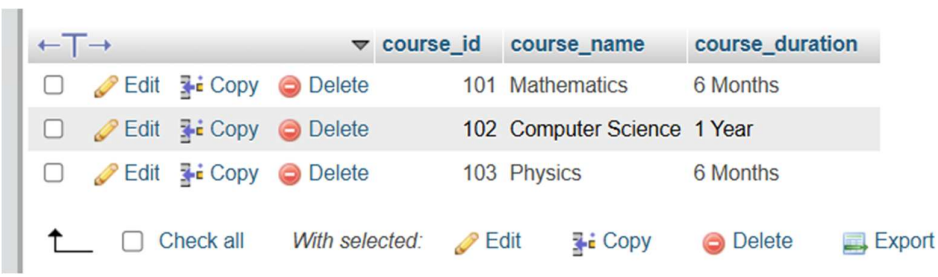


7. Data Manipulation Language (DML):

Lab 1: Insert three records into the courses table using the INSERT command.

Ans:

INSERT INTO courses (course_id, course_name, course_duration) VALUES
(101, 'Mathematics', '6 Months'),
(102, 'Computer Science', '1 Year'),
(103, 'Physics', '6 Months');



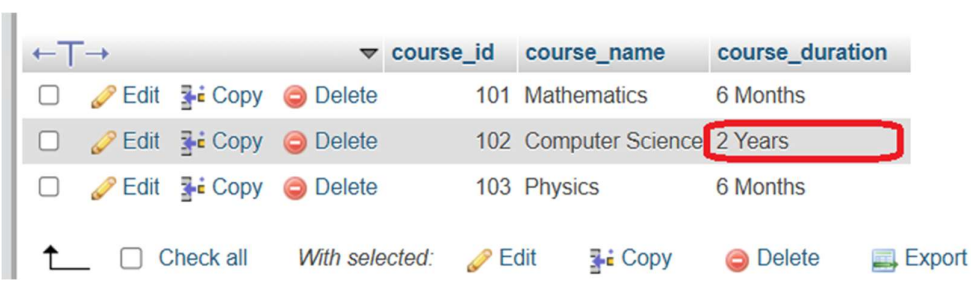
course_id	course_name	course_duration
101	Mathematics	6 Months
102	Computer Science	1 Year
103	Physics	6 Months

Check all With selected: Edit Copy Delete Export

Lab 2: Update the course duration of a specific course using the UPDATE command.

Ans:

UPDATE courses SET course_duration = '2 Years' WHERE course_id = 102;



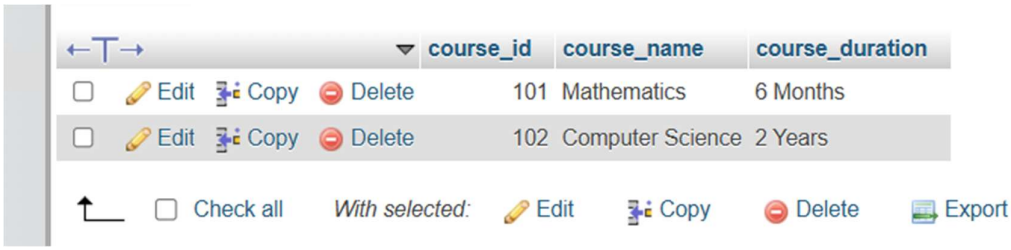
course_id	course_name	course_duration
101	Mathematics	6 Months
102	Computer Science	2 Years
103	Physics	6 Months

Check all With selected: Edit Copy Delete Export

Lab 3: Delete a course with a specific course_id from the courses table using the DELETE command.

Ans:

DELETE FROM courses WHERE course_id = 103;



8. Data Query Language (DQL):

Lab 1: Retrieve all courses from the courses table using the SELECT statement.

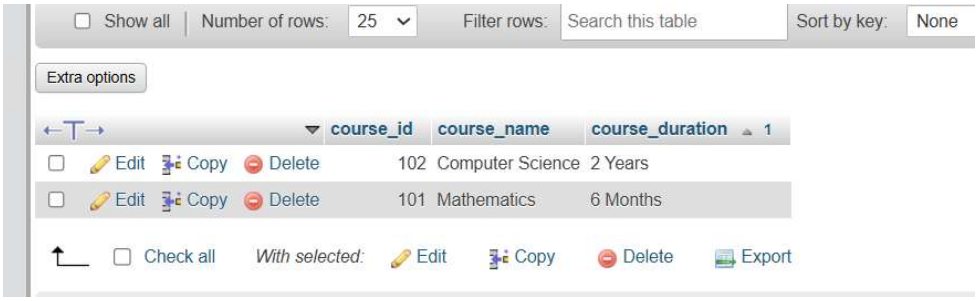
Ans:

SELECT * FROM courses;

Lab 2: Sort the courses based on course_duration in descending order using ORDER BY.

Ans:

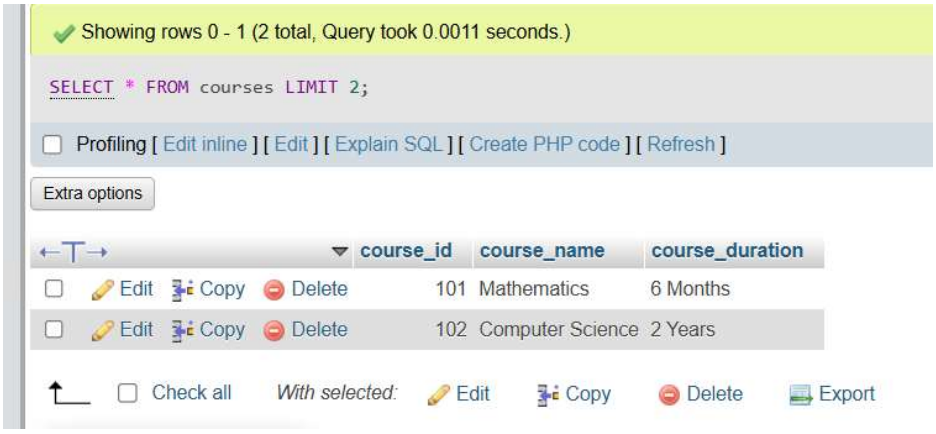
SELECT * FROM courses ORDER BY course_duration DESC;



Lab 3: Limit the results of the SELECT query to show only the top two courses using LIMIT.

Ans:

SELECT * FROM courses LIMIT 2;



9. Data Control Language (DCL):

Lab 1: Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table.

Ans:

Creating Users:

CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password1';

CREATE USER 'user2'@'localhost' IDENTIFIED BY 'password2';

Giving permissions:

GRANT SELECT ON school_db.courses TO 'user1'@'localhost';

Lab 2: Revoke the INSERT permission from user1 and give it to user2.

Ans:

```
REVOKE INSERT ON school_db.courses FROM 'user1'@'localhost';

GRANT INSERT ON school_db.courses TO 'user2'@'localhost';
```

10. Transaction Control Language (TCL):

Lab 1: Insert a few rows into the courses table and use COMMIT to save the changes.

Ans:

```
START TRANSACTION;

INSERT INTO courses (course_id, course_name, course_duration)
VALUES (201, 'Chemistry', '6 Months'),
(202, 'Biology', '1 Year');

COMMIT;
```

Lab 2: Insert additional rows, then use ROLLBACK to undo the last insert operation.

Ans:

```
START TRANSACTION;
INSERT INTO courses (course_id, course_name, course_duration)
VALUES (203, 'English', '6 Months'),
(204, 'History', '1 Year');
ROLLBACK;
```

Lab 3: Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

Ans:

```
START TRANSACTION;

SAVEPOINT before_update;

UPDATE courses
SET course_duration = '2 Years'
WHERE course_id = 201;

ROLLBACK TO before_update;

COMMIT;
```

11. SQL Joins:

Lab 1: Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

Ans:

```
-- Create tables :
CREATE TABLE departments (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(100)
);

CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   employee_name VARCHAR(100),
   department_id INT,
   FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

-- INNER JOIN :
```

```
SELECT e.employee_id, e.employee_name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Lab 2: Use a LEFT JOIN to show all departments, even those without employees.

Ans:

```
SELECT d.department_id, d.department_name, e.employee_name
FROM departments d
LEFT JOIN employees e
ON d.department_id = e.department_id;
```

12. SQL Group By:

Lab 1: Group employees by department and count the number of employees in each department using GROUP BY.

Ans:

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

Lab 2: Use the AVG aggregate function to find the average salary of employees in each department.

Ans:

```
SELECT department_id, AVG(salary) AS average_salary
FROM employees
GROUP BY department_id;
```

13. SQL Stored Procedure:

Lab 1: Write a stored procedure to retrieve all employees from the employees table based on department.

Ans:

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByDepartment(IN dept_id INT)
BEGIN
SELECT *
FROM employees
WHERE department_id = dept_id;
END //

DELIMITER ;
```

Lab 2: Write a stored procedure that accepts course_id as input and returns the course details.

Ans:

```
DELIMITER //

CREATE PROCEDURE GetCourseById(IN c_id INT)
BEGIN
SELECT *
FROM courses
WHERE course_id = c_id;
END //

DELIMITER ;
```

14. SQL View:

Lab 1: Create a view to show all employees along with their department names.

Ans:

```
CREATE VIEW employee_department_view AS
SELECT e.employee_id,
e.employee_name,
```

```
e.salary,
d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

Lab 2: Modify the view to exclude employees whose salaries are below $50,000.

Ans:

```
CREATE OR REPLACE VIEW employee_department_view AS
SELECT e.employee_id,
e.employee_name,
e.salary,
d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
WHERE e.salary >= 50000;
```

15. SQL Triggers:

Lab 1: Create a trigger to automatically log changes to the employees table when a new employee is added.

Ans:

Create log table:

```
CREATE TABLE employee_log (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  employee_id INT,
  action_type VARCHAR(50),
  action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Trigger to log new employee insertion:

```
DELIMITER //

CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
  INSERT INTO employee_log (employee_id, action_type)
  VALUES (NEW.employee_id, 'INSERT');
END //

DELIMITER ;
```

Lab 2: Create a trigger to update the last_modified timestamp whenever an employee record is updated.

Ans:

Add last_modified column (if not exists):

```
ALTER TABLE employees
ADD last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP;
```

Trigger to update last_modified on record update:

```
DELIMITER //

CREATE TRIGGER before_employee_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
  SET NEW.last_modified = CURRENT_TIMESTAMP;
END //
```

```
DELIMITER ;
```

16. Introduction to PL/SQL:

Lab 1: Write a PL/SQL block to print the total number of employees from the employees table.

Ans:

PL/SQL block to print total number of employees:

```
SET SERVEROUTPUT ON;

DECLARE
  total_emp NUMBER;
BEGIN
  SELECT COUNT(*) INTO total_emp
  FROM employees;

  DBMS_OUTPUT.PUT_LINE('Total Employees: ' || total_emp);
END;
/
```

Lab 2: Create a PL/SQL block that calculates the total sales from an orders table.

Ans:

PL/SQL block to calculate total sales from orders table:

```
SET SERVEROUTPUT ON;

DECLARE
  total_sales NUMBER;
BEGIN
  SELECT SUM(order_amount) INTO total_sales
  FROM orders;
```

```
  DBMS_OUTPUT.PUT_LINE('Total Sales: ' || total_sales);
END;
/
```

17. PL/SQL Control Structures:

Lab 1: Write a PL/SQL block using an IF-THEN condition to check the department of an employee.

Ans:

```
SET SERVEROUTPUT ON;
DECLARE
  v_dept_id employees.department_id%TYPE;
BEGIN
  SELECT department_id
  INTO v_dept_id
  FROM employees
  WHERE employee_id = 1;

  IF v_dept_id = 10 THEN
    DBMS_OUTPUT.PUT_LINE('Employee belongs to HR Department');
  ELSIF v_dept_id = 20 THEN
    DBMS_OUTPUT.PUT_LINE('Employee belongs to IT Department');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Employee belongs to Other Department');
  END IF;
END;
/
```

Lab 2: Use a FOR LOOP to iterate through employee records and display their names.

Ans:

```
SET SERVEROUTPUT ON;
BEGIN
  FOR emp_rec IN (SELECT employee_name FROM employees) LOOP
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.employee_name);
  END LOOP;
END;
/
```

18. SQL Cursors:

Lab 1: Write a PL/SQL block using an explicit cursor to retrieve and display employee details.

Ans:

```
SET SERVEROUTPUT ON;
DECLARE
  CURSOR emp_cursor IS
    SELECT employee_id, employee_name, salary
    FROM employees;

  v_id employees.employee_id%TYPE;
  v_name employees.employee_name%TYPE;
  v_salary employees.salary%TYPE;

BEGIN
  OPEN emp_cursor;

  LOOP
    FETCH emp_cursor INTO v_id, v_name, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;
```

```
      DBMS_OUTPUT.PUT_LINE('ID: ' || v_id ||
                    ', Name: ' || v_name ||
                    ', Salary: ' || v_salary);
   END LOOP;

   CLOSE emp_cursor;
END;
/
```

Lab 2: Create a cursor to retrieve all courses and display them one by one.

Ans:

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR course_cursor IS
      SELECT course_id, course_name, course_duration
      FROM courses;

   v_cid courses.course_id%TYPE;
   v_cname courses.course_name%TYPE;
   v_duration courses.course_duration%TYPE;

BEGIN
   OPEN course_cursor;

   LOOP
      FETCH course_cursor INTO v_cid, v_cname, v_duration;
      EXIT WHEN course_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE('Course ID: ' || v_cid ||
```

```
                    ', Name: ' || v_cname ||
                    ', Duration: ' || v_duration);
    END LOOP;

    CLOSE course_cursor;
END;
/
```

19. Rollback and Commit Savepoint:

Lab 1: Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

Ans:

```
START TRANSACTION;

SAVEPOINT sp1;

INSERT INTO courses (course_id, course_name, course_duration)
VALUES (301, 'Economics', '6 Months');

INSERT INTO courses (course_id, course_name, course_duration)
VALUES (302, 'Statistics', '1 Year');

ROLLBACK TO sp1;

COMMIT;
```

Lab 2: Commit part of a transaction after using a savepoint and then rollback the remaining changes.

Ans:

```
START TRANSACTION;

INSERT INTO courses (course_id, course_name, course_duration)
VALUES (303, 'Philosophy', '6 Months');

SAVEPOINT sp2;

INSERT INTO courses (course_id, course_name, course_duration)
VALUES (304, 'Sociology', '1 Year');

ROLLBACK TO sp2;

COMMIT;
```